

# TSFS12 HAND-IN EXERCISE 4

## Collaborative Control

August 26, 2025

### 1 OBJECTIVE

The objective of this hand-in exercise is to explore and implement the methods described in Lecture 8:

- Position-based formation control
- Displacement-based formation control
- Distance-based formation control

The multi-agent formation control problem will in each case be solved for a set of agents with a desired formation, and the result is a model in the form

$$\begin{aligned}\dot{x}_i &= f_i(t, x_i, u_i) \\ y_i &= h_i(x) \\ u_i &= g_i(y_i)\end{aligned}$$

$i = 1, \dots, N$ , where  $N$  is the number of agents,  $x_i$  is the state vector of the agent,  $y_i$  is the measurements available to the agent,  $u_i$  is a control signal, and

$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_N \end{pmatrix}$$

is the state vector of the complete system.

### 2 PREPARATIONS BEFORE THE EXERCISE

Before doing this exercise, make sure you understand the material covered in

- Lecture notes on Collaborative control (Lecture 8). The notation and terminology from these lecture notes will be used in this document.
- Oh, Kwang-Kyu, et al. "A survey of multi-agent formation control. *Automatica* 53 (2015) 424–440.

To get the background code for the exercise, get the latest files from <https://gitlab.liu.se/vehsys/tsfs12> and familiarize yourself with the provided code.

You are free to choose in which language to implement your planners, there is skeleton code available in Matlab and Python. In the student labs, all Python packages needed are pre-installed in the virtual environment activated by

```
source /courses/tsfs12/env/bin/activate
```

### 3 REQUIREMENTS

To complete this exercise you should implement problems described in Section 5. The exercise is examined by submitting the following on the Lisam course page.

1. Runnable code. If your code consists of several files, submit a zip-archive with your files. Do not include files for the language that you did not use.
2. A report
  - that answers the questions, including relevant plots, in Section 5, and other questions that you have encountered during solving this exercise
  - important to include your reflections in the document, and not just report results and code. It is necessary to include a discussion whenever requested in the questions.
  - includes a concluding discussion

The report is expected to be a neat, self-contained, document in PDF format. You are encouraged, but not required, to use the templates provided at the course git-repo.

It is allowed to discuss the exercises on a general level with other course participants. However, code sharing outside groups is not allowed. Moreover, both students in the group should be fully involved in performing all exercises, and thereby be prepared to answer questions on all subtasks.

See Appendix 6 for the extra assignment needed for higher grades. The extra assignment is done individually and is submitted by a document answering the questions, including suitable figures, plots, and tables. There is only pass/fail on the extra exercise and the document can not be revised or extended after submission. It is not required to get everything correct to pass the assignment, we assess the whole submission. You are of course welcome to ask us if you have questions or want us to clarify the exercise.

Since these exercises examine this course, we would like to ask you not to distribute or make your solutions publicly available, e.g., by putting them on a public GitHub repository. A private repository on, e.g., gitlab@liu is of course fine.

Thanks for your assistance!

### 4 CONTROLLER IMPLEMENTATION ENVIRONMENT

To define the simulation environment, we need to define the following components:

- a system model for each agent
- a control function for each agent
- a graph specifying which information is made available to each agent
- a set of reference functions for each agent in the formation

Usage of the provided code is illustrated by example. A small 4-agent example is provided `ex0.{m,ipynb,py}`, where a single-integrator model  $\dot{p}_i = u_i$  is implemented for each agent, with absolute position measurements  $y_i = p_i$  and the control law  $u_i = k_p(p_i^* - p_i)$ . Here  $p_i^*$  represents the reference position for the agent  $i$ . The corresponding system-model function, a single integrator, is defined as:

Matlab

```

1 function dxdt =
2   single_integrator(t, x, u, mdlpar)
3   dxdt = u;
4 end

```

Python

```

1 def single_integrator(t, x, u, mdlpar):
2   return u

```

where  $t$  is the current time,  $x$  is the state vector of the agent,  $u$  is the control input, and `mdlpar` is an object with any model parameters. In this case, the model has no parameters and thus the `mdlpar` variable is not used.

Further, the control function based on the measurement  $y$  is defined as:

Matlab

```

1 function u = g_absolute(y, yref, ctrlpar)
2   u = -ctrlpar.k * (y - yref);
3 end

```

Python

```

1 def g_absolute(y, yref, ctrlpar):
2   Kp = ctrlpar["k"]
3   return -Kp * (y - yref)

```

where  $y$  is the measurement,  $yref$  is the reference position, and `ctrlpar` are the set of control parameters.

We will also need to define formation graphs for the multi-agent systems. The graph defines relations between the agents and defines the measurements for each agent. The graph is specified by a set of edges and an edge  $(i, j)$  represents that agent  $j$  measures the relative information, e.g., distance, velocity, from agent  $i$ .

In a simple case where each agent  $i$  measures its own absolute position, this is represented by a tuple with a single element  $i$  as  $(i)$ . Defining that all 4 agents measures its own absolute position is then specified as:

Matlab

```

1 G = {[1,], ... % Agent 1
2       [2,], ... % Agent 2
3       [3,], ... % Agent 3
4       [4,], ... % Agent 4
5   };

```

Python

```

1 G = [(0,), # Agent 0
2       (1,), # Agent 1
3       (2,), # Agent 2
4       (3,), # Agent 3
5   ]

```

The formation references for each agent are specified as a list of functions, where each function returns the reference for the agent at a given time. For example, in the provided example, the reference positions are defined as (note the transposes in the Matlab code):

Matlab

```

1 formation_references = {
2   @(t) [cos(2*t), sin(2*t)]';
3   @(t) [2*cos(-t), 2*sin(-t)]';
4   @(t) [3, 2]';
5   @(t) [5, 4]';
6 };

```

Python

```

1 formation_references = [
2   lambda t: [np.cos(2 * t), np.sin(2 * t)],
3   lambda t: [2 * np.cos(-t), 2 * np.sin(-t)],
4   lambda t: [3, 2],
5   lambda t: [5, 4],
6 ]

```

Here we want the agent 1 to move counter-clockwise with radius 1, such that its reference positions are defined as the vector  $(\cos(2t), \sin(2t))$ . The references of other agents are defined similarly.

Finally, we compile everything in a single object that we can simulate. Therefore, based on the defined system models, control functions, the system graph, and the set of reference functions, the complete multi-agent system is created as:

```

1 agent = [
2     CreateAgent(@single_integrator, modelparam, @g_absolute, controlparam2), ...
3     CreateAgent(@single_integrator, modelparam, @g_absolute, controlparam1), ...
4     CreateAgent(@single_integrator, modelparam, @g_absolute, controlparam1), ...
5     CreateAgent(@single_integrator, modelparam, @g_absolute, controlparam2), ...
6 ];
7
8 formation = AgentFormation(agent, G, formation_references, n);

```

The object `formation` holds all information about the multi-agent system, and the full state vector of the formation is the concatenation of the state vectors of all agents. For example, in line 2 it creates agent 1, which is modeled by a `single_integrator` and computes the control input by `g_absolute`, with the model and control parameters are included in the structs `modelparam` and `controlparam1`, respectively. Then, the agent graph and reference functions are defined for the multi-agent system through `AgentFormation`, where `n` is the number of states for each agent (2 in this case). The size of the state-vector must, currently, be the same for all agents.

In Python, this is correspondingly defined as:

```

1 agents = [
2     CreateAgent(single_integrator, modelparam, g_absolute, ctrl_params_2),
3     CreateAgent(single_integrator, modelparam, g_absolute, ctrl_params_1),
4     CreateAgent(single_integrator, modelparam, g_absolute, ctrl_params_1),
5     CreateAgent(single_integrator, modelparam, g_absolute, ctrl_params_2)]
6
7 formation = AgentFormation(agents, G, formation_references, n)

```

With the `formation` object, the system of differential equations

$$\dot{x}_{\text{sys}} = f_{\text{sys}}(t, x_{\text{sys}})$$

can be accessed using the `formation.ode` function. Then, the multi-agent system can be simulated using standard solvers, e.g., `ode23t` in Matlab or `scipy.odeint` in python as:

Matlab

```
1 [t, x] = ode23t(@formation.ode, tspan, xo);
```

Python

```
1 x = odeint(formation.ode, xo, t)
```

The initial state of the multi-agent system is `x0`, the simulator outputs `x` as the state vector of the system containing the states of all agents at every time step in the simulation. Code is provided that plots and animates the formation behavior.

## 5 EXERCISES

This section summarizes a number of discussion topics and questions you should implement, investigate, and reflect upon to pass this exercise. Hand in one separate file for each assignment, and a brief report discussing your results.

### 5.1 Position-based formation control

**Exercise 5.1.** Based on the example in `ex0.m` you need to do several modifications. Firstly, the single-integrator model in `ex0.m` is replaced with a double-integrator

$$\ddot{p}_i = u_i,$$

the control law

$$u_i = k_v(v_i^* - v_i) + k_p(p_i^* - p_i)$$

should be used in this exercise, here the position  $p_i$  and velocity  $v_i$  are measured. Add a constant force, caused by a wind, in the  $x$ -direction to the model, and simulate the model (hint: the force can be regarded as the extra control input).

Copy `ex0.{m,ipynb,py}` and rename it as `ex_5_1.{m,ipynb,py}`, then implement your controller within this script and analyze the results by looking at the animation; in the report you should also plot the trajectories of the agents with and without wind force.

Hint: to implement this controller based on `ex0.{m,ipynb,py}`, you need to consider changing the agent graph, the reference graph, model and control parameters, number of states, initial state, the system-model function, and the control function. You are not expected to change all these items, but these parts cover all possible operations you may have.

## 5.2 Displacement-based formation control

**Exercise 5.2.** In this exercise, there are 8 agents modelled with a double integrator  $\ddot{p}_i = u_i$ . The desired formation is as vertical string with relative distance one and relative velocity zero. There are two types of agents in the system. The agent at the top of the string that can measure its absolute position and the objective is to reach the point  $(0,4)$ . The other agents can measure the relative displacement to their two nearest neighbours, and the objective for them is to keep the same relative displacement as the desired string formation.



It is assumed that the absolute positions  $p_i$  and velocities  $v_i$  can be measured by the first agent and the control law

$$u_i = k_v(v_i^* - v_i) + k_p(p_i^* - p_i)$$

is used for this agent. The control law

$$u_i = k_p \sum_{j \in \mathcal{N}_i} (p_j - p_i - p_j^* + p_i^*) + k_v \sum_{j \in \mathcal{N}_i} (v_j - v_i - v_j^* + v_i^*),$$

is used for the other agents, where the relative states  $p_j - p_i$  and  $v_j - v_i$  are measured by these agents.  $\mathcal{N}_i$  is the set of neighbours to agent  $i$ .

(1). Based on your previous implementations, create a new file `ex_5_2_1.{m,ipynb,py}` for this exercise, simulate the system with  $k_p = k_v = 2$ , and with initial position  $(i, 0)$  and initial velocity equal to zero for agent  $i$ ,  $i = 1, \dots, 8$ . In this simulation, fix the desired position of agent 1 as  $(0, 4)$ .

(2). Create a new file `ex_5_2_2.{m,ipynb,py}` for this exercise, remove all edges from node  $i - 1$  to node  $i$ ,  $i = 3, \dots, 8$ , i.e., all measurements of the relative state

of the agent below are removed. The desired position of agent 1 was previously fixed to  $(0, 4)$ . Add a harmonic oscillation in the  $y$ -direction with amplitude 0.2 to this position and simulate the system with angular frequencies  $1 \text{ rad/s}$  and  $2 \text{ rad/s}$ . Choose the simulation time large enough so that the formation is located on a vertical line at the end of the simulation.

(3). Analyze the results by looking at the animations of these two cases. In the report, include the plot of the vertical velocities for the agents for the two frequencies. Describe how the behavior of the agents differs between the two cases.

(4). You may have noticed that the control law for the first agent that measures its global state is defined using  $v_i^* - v_i$  and  $p_i^* - p_i$  (state reference minus state), while the control law for the rest of agents is defined using  $v_i - v_i^*$  and  $p_i - p_i^*$  (state minus state reference). Try to change the sign of the control laws in this case and discuss the reasons for the corresponding results. If you do not know the reason, report what you have found.

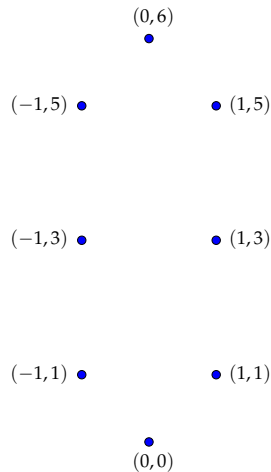
Hint:

- You need to consider possible modifications of the agent graph, the reference graph, model and control parameters, number of states, initial state, the system-model function, and the control function.
- Here we have two types of agents, one implements the control single based on global measurement and one based on relative measurement, so you need to implement two types of control functions for those agents.
- When you define the control function for the agent that uses relative measurement, note that the measurement contains the relative states from the neighbors of this agent, therefore, there may need a forloop to go through all relative measurements to compute the control single, as indicated by the control-law equation.

### 5.3 Distance-based formation control

#### Exercise 5.3.

The desired formation in this case is given by the figure below



The first step is to construct a minimally rigid graph for the nodes. Note that a single-integrator model

$$\dot{p}_i^i = u_i^i, \quad i = 1, \dots, N$$

will be used in this exercise, where  $p_i^i$  and  $u_i^i$  denote the position and control signal with respect to the local coordinate system, as described in Lecture 8. It is assumed that the agent at the top can measure its absolute position, and it will use the control law

$$u_i = k_p(p_i^* - p_i)$$

The control signal for the other agents is computed using a gradient control law:

$$u_i^i = -\nabla_{p_i^i} \phi = k_p \sum_{j \in \mathcal{N}_i} \gamma'_{ij}(\|p_j^i - p_i^i\|) \frac{p_j^i - p_i^i}{\|p_j^i - p_i^i\|}$$

The local potential function is defined by

$$\phi_i = \frac{k_p}{2} \sum_{j \in \mathcal{N}_i} \gamma_{ij}(\|p_j^i - p_i^i\|)$$

with  $\gamma_{ij}$  defined as

$$\gamma_{ij}(\|p_j - p_i\|) = (\|p_j - p_i\|^2 - \|p_j^* - p_i^*\|^2)^2$$

(1). Based on your previous implements, create a new file `ex_5_3_1.{m,ipynb,py}`, assume that the desired position of the agent at the top is moving on a circle of radius 2. Simulate the model in this case with  $k_p = 1$  and  $k_p = 5$ , and plot the distance between the top node and one of its neighbors in the report. Explain the difference.

(2). Create a new file `ex_5_3_2.{m,ipynb,py}`, it is assumed that the agent at the bottom also can measure absolute position. Use this additional information to control the agents so the orientation of the formation is preserved. Plot the paths of the agents in the report.

## 6 EXTRA ASSIGNMENT

The extra assignment has two parts: Exercise 6.1 and 6.2.

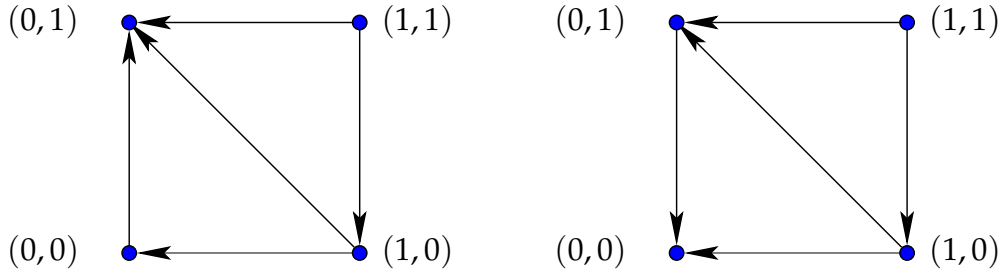
### 6.1 Measurement errors

**Exercise 6.1.** Replace the control law for agent at the top in Exercise 5.3 with the same gradient control law as the other agents. The agent at position  $(1,3)$  has an error in the measurement of the distance to the agent at position  $(1,5)$  and the measurement is 20% larger than the correct value. Add this measurement error to the model and simulate the system.

- Describe the consequences of the measurement error and explain why the formation behaves the way it does.
- Give an example where two measurement errors make the formation move in the  $y$ -direction.
- Give an example where two measurement errors make the agents rotate around the center of the formation.

### 6.2 Distance based directed formation control

**Exercise 6.2.** In Exercise 5.3 and 6.1, distance based formation control was used with undirected graphs that specified the set of neighbours for each agent. In this exercise we shall use distance based formation control with directed graphs. Consider the following two directed graphs with vertices at the corners of the unit square:



As in Exercise 5.2 this will define the desired formation and the direction of the arrows defines which distance measures that are available for each agent. For example, the agent located at position  $(0,1)$  in the graph to the left can measure the distance to the other three agents.

Use the same gradient control law as before for the four agents and simulate the system with measurements available as represented by one of the two graphs at a time. Try some different initial conditions and describe the formation the agents converge to. Explain why the results are different for the two graphs.